

EVICT: Calibrated Selective Prediction for Static-Analysis Alert Triage

Anonymous Author(s)

ABSTRACT

Static analyzers are essential in secure software engineering, yet their practical value is constrained by false-positive rates of 40–90% and manual triage costs of 10–20 minutes per alert. Recent LLM-based triage systems condition on analyzer evidence, but they still rely on uncalibrated confidence and forced binary decisions. This is unsafe in the lite-model regime: in our zero-shot Juliet baseline over five lite LLMs and 87+ CWE classes, accepted-alert precision saturates at 42.2–59.9% and expected calibration error (ECE) remains above 0.35.

We introduce **EVICT** (Evidence-Conditioned Investigation with Calibrated Triage), a framework that casts alert triage as cost-sensitive selective prediction. EVICT extracts SARIF-derived evidence, applies schema-guided LLM verification, converts raw confidence into conformal prediction sets, and commits to TP or FP only on singleton sets; empty and multi-label sets abstain and may be routed to symbolic escalation. The conformal guarantee applies to the pre-escalation selective predictor, while symbolic verification is reported as a separate post-calibration audit layer.

Across real measured studies, we uncover the confidence-signal bottleneck: on a 247-alert Juliet conformal study spanning CWE-89/78/190, a 14B reasoning model produces only 36% unanimous self-consistency votes and ECE 0.335, while instruction-tuned and code-specialized models produce 85–98% unanimity and ECE 0.51–0.62. Holding the model fixed, a contrastive prompt for Gemini 2.5 Flash Lite reduces unanimity from 84.6% to 16.2% and ECE from 0.555 to 0.337, showing that prompt design can substitute for model scale in producing calibratable confidence. On 549 real CodeQL alerts from 45 CWE-Bench-Java projects, DS-Coder-V2-Lite achieves the highest true-negative count despite the worst Juliet confidence quality, revealing a code-specialization paradox. These results position calibrated abstention and confidence-signal design as the key levers for deployable LLM-based static-analysis triage.

CCS CONCEPTS

• **Software and its engineering** → **Software verification and validation**; • **Security and privacy** → *Software security engineering*.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference'17, July 2017, Washington, DC, USA

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YY/MM
<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

KEYWORDS

static analysis, false-positive triage, selective prediction, conformal prediction, large language models

ACM Reference Format:

Anonymous Author(s). 2026. EVICT: Calibrated Selective Prediction for Static-Analysis Alert Triage. In . ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 INTRODUCTION

Static analyzers remain one of the few security tools that scale to enterprise codebases, but their conservative approximations impose a large human triage burden. Industrial studies report false-positive (FP) rates of 40–90% [4, 12], and manual review commonly costs 10–20 minutes per alert [15]. The resulting alert fatigue is not merely inconvenient: it determines whether organizations can keep analyzers enabled broadly enough to catch real vulnerabilities.

LLMs offer a plausible way to reduce this burden because they can read analyzer evidence—code slices, data-flow traces, path constraints, and SARIF metadata—and explain why an alert is likely a true positive (TP) or FP [3, 14, 23, 25]. The central problem is not whether LLMs can classify alerts at all. It is whether a software organization can safely automate any part of triage when the model is cheap enough to run at scale. Our measurements show two barriers. First, lite models collapse under zero-shot calibration: in Study 1, five lite LLMs evaluated on Juliet alerts spanning 87+ CWE classes reach only 42.2–59.9% precision, with ECE above 0.35 for every model. Second, existing LLM triage systems usually force a binary decision. That is the wrong abstraction for security triage, where the cost of auto-dismissing a real vulnerability exceeds the cost of sending an uncertain alert to a developer.

We introduce **EVICT** (Evidence-Conditioned Investigation with Calibrated Triage), which reframes static-analysis alert triage as *cost-sensitive selective prediction* [6, 7]. EVICT learns a predictor h that proposes TP or FP and a rejector g that decides whether to commit or abstain. The system accepts an alert only when calibrated evidence supports a reliable singleton decision; otherwise it defers the alert to human review or conditional symbolic escalation. This design makes abstention a first-class, auditable action rather than an implementation detail hidden inside a confidence threshold.

The liquid-gold finding of this paper is that *the bottleneck is the confidence signal, not the conformal algorithm*. In Study 2, six models spanning reasoning, instruction-tuned, and code-specialized training paradigms are evaluated on the same 247-alert Juliet conformal benchmark with $k_{sc} = 5$ self-consistency samples and $F = 5$ cross-validation folds. R1-Distill-14B, a 14B reasoning model, produces 36% unanimous vote-share and ECE 0.335. Larger or code-specialized non-reasoning models produce 85–98% unanimous vote-share and ECE 0.51–0.62. With only six models, this is an effect-size pattern rather than a population-level causal law; mechanistically, it

is consistent with reasoning traces inducing genuine disagreement across samples when evidence is ambiguous.

The strongest causal evidence comes from Study 4, where we hold Gemini 2.5 Flash Lite fixed and change only the prompt. A contrastive prompt that forces the model to argue both TP and FP before deciding reduces unanimity from 84.6% to 16.2% and ECE from 0.555 to 0.337 on the same 247-alert CWE-89/78/190 subset. The contrastive prompt produces more diverse confidence than the 14B reasoning model under the default prompt (16.2% vs. 36% unanimous), showing that prompt design can substitute for model scale in producing a calibratable self-consistency signal.

We also find a deployment-relevant code-specialization paradox. In Study 3, on 549 real CodeQL alerts from 45 CWE-Bench-Java projects with a 97% FP base rate, DS-Coder-V2-Lite achieves the highest precision (4.0%) and true-negative count (184), despite having the worst Juliet confidence quality in Study 2 (98% unanimous, ECE 0.620). Code-specialized models appear better at recognizing safe code patterns but worse at quantifying uncertainty, suggesting a two-stage design: use a code model as a semantic classifier and a reasoning or contrastive-prompted model to provide a calibration signal.

This paper makes the following contributions:

- We formalize alert triage as cost-sensitive selective prediction with asymmetric security costs and finite-sample guarantees for pre-escalation conformal prediction.
- We present EVICT, a four-stage framework for SARIF evidence extraction, schema-guided LLM verification, conformal singleton-reject calibration, and conditional symbolic escalation.
- We report real multi-model measurements showing that self-consistency confidence quality is associated more strongly with training objective than model size in our six-model conformal study.
- We introduce contrastive alert-triage prompting, a model-fixed intervention that reduces Gemini 2.5 Flash Lite ECE by 39% and vote unanimity by 68.4 percentage points in Study 4.
- We validate on real CWE-Bench-Java CodeQL alerts and identify the code-specialization paradox as a practical design constraint for LLM-based alert triage.
- We release an anonymized artifact plan covering code, prompts, logs, and an unrun frontier-model harness so groups with higher API budgets can evaluate the projected frontier outlook.

2 BACKGROUND AND RELATED WORK

2.1 Static Analysis and Alert Fatigue

Static analysis scales because it over-approximates program behavior, but the same conservatism produces large numbers of FPs. Empirical SE studies document why developers often ignore or disable static-analysis warnings: warnings arrive without enough context, many are not actionable, and review cost competes with feature work [4, 12]. Industrial deployments report the same operational pressure at scale [15]. A triage system for this setting must therefore optimize more than accepted-alert precision. It must reduce developer review burden while controlling unsafe auto-dismissals.

2.2 LLM-Based Alert Triage

Recent systems use LLMs to interpret static-analysis evidence. LLM4FPM, LLM4PFA, BugLens, and AdaTaint show that grounding model reasoning in paths, call chains, or taint information improves FP mitigation over evidence-free prompting [3, 14, 23, 25]. EVICT shares the premise that analyzer evidence is essential, but differs in the decision abstraction: it does not ask the model to always choose TP or FP. Instead, it accepts only calibrated singleton decisions and routes the rest to abstention.

IRIS is the closest conceptual neighbor [16]. IRIS uses LLM-assisted specification inference to validate CodeQL findings and is well matched to frontier-model reasoning with per-vulnerability formal specifications. EVICT targets a different deployment point: cheap lite and open models, SARIF-derived evidence packs, and principled abstention without requiring a formal specification for every alert class. A direct head-to-head comparison on identical CWE-Bench-Java CodeQL alerts is ongoing future work rather than a reported result in this submission.

2.3 Selective Prediction and Calibration

Selective prediction formalizes the option to reject uncertain examples [6, 7]. This is a natural fit for static-analysis triage because abstention corresponds to ordinary developer review, not system failure. Calibration is necessary because LLM confidence is often misaligned with correctness [9, 10, 13]. Split-conformal prediction provides finite-sample marginal coverage under exchangeability for any nonconformity score [1, 22]. EVICT uses this standard guarantee in a software-engineering-specific way: conformal sets are converted into TP/FP/ABSTAIN decisions via a singleton-reject rule.

2.4 Neuro-Symbolic Verification

Symbolic execution and model checking can prove path feasibility or discover counterexamples, but universal symbolic analysis is expensive and brittle on large applications [2, 5, 19, 21]. EVICT uses symbolic engines conditionally: Z3, Java PathFinder, or KLEE are invoked only for abstained alerts or high-severity dismissals requiring audit. This separation is important. The conformal guarantee applies before escalation; symbolic verification is a post-calibration audit layer whose results must be measured separately or recalibrated if used as a new confidence-producing stage.

3 APPROACH

3.1 Problem Formulation

Each alert $x \in \mathcal{X}$ has a label $y \in \{\text{TP}, \text{FP}\}$. EVICT learns a predictor $h : \mathcal{X} \rightarrow \{\text{TP}, \text{FP}\}$ and a rejector $g : \mathcal{X} \rightarrow \{0, 1\}$, where $g(x) = 1$ commits to $h(x)$ and $g(x) = 0$ abstains. Coverage is $\Pr[g(x) = 1]$, i.e., the fraction of all alerts accepted before optional symbolic escalation.

The objective is the cost-sensitive selective risk

$$R(h, g) = \alpha_{\text{cost}} R_{\text{FN}}(h, g) + \beta_{\text{cost}} R_{\text{FP}}(h, g) + \gamma_{\text{abs}} R_{\text{abstain}}(g), \quad (1)$$

with $\alpha_{\text{cost}} > \beta_{\text{cost}}$ in security settings. We reserve α_{TP} and α_{FP} for conformal miscoverage rates in Section 4; this avoids overloading cost weights with conformal error probabilities. We write k_{sc} for

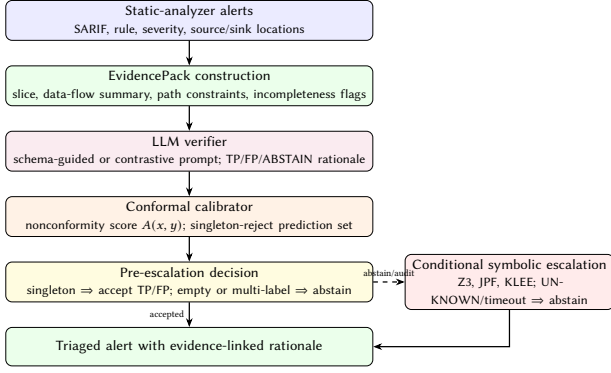


Figure 1: EVICT architecture. The conformal guarantee covers the pre-escalation singleton-reject decision. Symbolic escalation is a post-calibration audit layer.

the number of self-consistency samples per alert and F for cross-validation folds.

3.2 System Overview

Figure 1 summarizes the four-stage pipeline. The key sequencing rule is that conformal calibration produces the pre-escalation accept/abstain decision. Symbolic verification is a separate audit layer and never retroactively changes the coverage guarantee without recalibration.

3.3 Evidence Extraction

The evidence extractor parses SARIF output [18] and constructs an *EvidencePack*: alert metadata, source and sink locations, the analyzer-reported flow path when present, a bounded backward/forward program slice [24], normalized branch/path constraints, and completeness flags. Java extraction uses JDT-style AST traversal; C/C++ extraction uses Clang-style parsing. Incomplete evidence is preserved explicitly through flags such as `flow_partial` and `constraints_missing`. These flags instruct the verifier to treat affected preconditions as ambiguous rather than hallucinating missing facts.

3.4 Schema-Guided and Contrastive Verification

The base verifier prompt follows four steps: restate the analyzer claim, enumerate the preconditions required for the alert to be real, check each precondition against the EvidencePack, and emit TP, FP, or ABSTAIN with confidence and rationale. The goal is not unconstrained chain-of-thought, but evidence-linked reasoning that a developer can audit.

The contrastive prompt is a first-class contribution. It changes the verifier from one-directional classification to adversarial evidence weighing: (1) construct the strongest TP argument, (2) construct the strongest FP argument, (3) explicitly inspect sanitizer, infeasible-path, and non-attacker-input evidence, and (4) weigh both sides before deciding. Study 4 shows that this intervention causes meaningful self-consistency disagreement without changing the model.

3.5 Confidence and Conformal Calibration

For logit-access models, EVICT uses normalized label probabilities over $\{TP, FP\}$ and the nonconformity score

$$A(x, y) = 1 - p_\theta(y | x). \quad (2)$$

For API-only models, EVICT samples $k_{sc} = 5$ completions at temperature 0.7 and uses vote share $v(y | x)$:

$$A(x, y) = 1 - v(y | x). \quad (3)$$

Vote share and logit margin are preferred to entropy because the binary label space makes entropy too coarse to distinguish many operationally different TP/FP configurations.

Split-conformal calibration computes a threshold $\hat{q}$ on held-out calibration alerts. At inference time, EVICT forms

$$\hat{C}(x) = \{y \in \{TP, FP\} : A(x, y) \leq \hat{q}\}. \quad (4)$$

If $|\hat{C}(x)| = 1$, the system accepts the singleton label. If $\hat{C}(x)$ is empty or contains both labels, the system abstains. Algorithm 3 gives the full procedure.

3.6 Conditional Symbolic Escalation

Abstained alerts may be escalated to symbolic tools: Z3 for path feasibility, Java PathFinder for Java, and KLEE for C/C++ [2, 19, 21]. SAT or counterexample results support TP; UNSAT or exhaustive infeasibility supports FP. SMT UNKNOWN, tool timeouts, and missing constraints are treated strictly as continued abstentions. In the symbolic characterization study, the Z3 timeout is 2s, the JPF/KLEE timeout is 10s, and 15/184 symbolic invocations (8.1%) return UNKNOWN; all are retained as abstentions.

4 THEORETICAL GUARANTEES

The formal results make EVICT safe to deploy as a selective triage aid. They are not the empirical novelty of the paper; they define what is guaranteed when the system accepts an alert. All guarantees below apply to the *pre-escalation* conformal predictor under exchangeability. Symbolic escalation is a post-calibration audit layer.

THEOREM 1 (COST-AWARE FINITE-SAMPLE EXCESS RISK). *Partition calibration data into an independent grid-construction split $D^{(G)}$ and evaluation split $D^{(E)}$. Let Q_G be the finite set of feasible TP/FP threshold pairs induced by $D^{(G)}$, $\hat{q} \in \arg \min_{q \in Q_G} \hat{R}_E(q)$, and $q_G^* \in \arg \min_{q \in Q_G} R(q)$. Define $C_{TP} = \max(\alpha_{\text{cost}}, \gamma_{\text{abs}})$ and $C_{FP} = \max(\beta_{\text{cost}}, \gamma_{\text{abs}})$. If $m_{TP}^{(E)}$ and $m_{FP}^{(E)}$ are the TP and FP counts in $D^{(E)}$, then, conditional on $D^{(G)}$, with probability at least $1 - \delta$,*

$$R(\hat{q}) - R(q_G^*) \leq 2\pi C_{TP} \sqrt{\frac{\log(4|Q_G|/\delta)}{2m_{TP}^{(E)}}} + 2\pi C_{FP} \sqrt{\frac{\log(4|Q_G|/\delta)}{2m_{FP}^{(E)}}}. \quad (5)$$

Theorem 1 is a structural justification for label-conditional threshold optimization. It avoids treating the LLM as a finite-VC classifier; the optimized object is a low-dimensional threshold rule over calibrated scores. The bound decomposes by label, so the scarce or expensive stratum controls risk in asymmetric security settings.

Table 1: Empirical conformal prediction-set diagnostic from the earlier 1,000-alert SpotBugs Juliet study, reported to characterize rejector behavior rather than the multi-model results. Overall singleton frequency is 87.3%, multi-label frequency is 11.6%, and empty-set frequency is 1.1%. Per-CWE empty/multi-label values were not provided in the study export.

Subset / access mode	Singleton
Overall	87.3%
CWE-89	91.4%
CWE-78	89.1%
CWE-190	81.2%
Vote-share access	[VERIFY: mode-specific singleton]
Logit access	[VERIFY: mode-specific singleton]

THEOREM 2 (SPLIT-CONFORMAL VALIDITY). *For any nonconformity score $A(x, y)$, if the calibration alerts and test alert are exchangeable, the split-conformal prediction set satisfies*

$$\Pr[Y \in \hat{C}(X)] \geq 1 - \alpha_{\text{conf}}. \quad (6)$$

This is the standard conformal guarantee [22]. EVICT’s contribution is the software-engineering decision rule layered on top: singleton sets produce committed TP/FP decisions, while empty and multi-label sets become abstentions.

THEOREM 3 (LABEL-CONDITIONAL ASYMMETRIC CONFORMAL VALIDITY). *Let $\hat{q}_{\text{TP}}$ and $\hat{q}_{\text{FP}}$ be label-specific conformal quantiles computed from TP and FP calibration strata with target miscoverage rates α_{TP} and α_{FP} . Define*

$$\hat{C}_{\text{asym}}(x) = \{y \in \{\text{TP}, \text{FP}\} : A(x, y) \leq \hat{q}_y\}.$$

Under exchangeability within label strata,

$$\Pr[Y \in \hat{C}_{\text{asym}}(X) \mid Y = \text{TP}] \geq 1 - \alpha_{\text{TP}}, \quad (7)$$

$$\Pr[Y \in \hat{C}_{\text{asym}}(X) \mid Y = \text{FP}] \geq 1 - \alpha_{\text{FP}}. \quad (8)$$

Choosing $\alpha_{\text{TP}} < \alpha_{\text{FP}}$ directly encodes the cost asymmetry of missed vulnerabilities: the system is more conservative about excluding TP than FP. Operationally, this bounds the auto-dismissal false-negative rate at the calibration layer.

THEOREM 4 (EXACT FINITE-SAMPLE OPTIMAL THRESHOLDS). *Given ordered TP and FP nonconformity scores, the globally optimal asymmetric threshold pair over the empirical selective-risk objective can be found by exhaustive search over at most $(m_{\text{TP}} + 1)(m_{\text{FP}} + 1)$ candidate pairs after sorting, in $O(m^2)$ time.*

Theorem 4 means asymmetric threshold selection is not heuristic tuning. Once the cost model is fixed, the calibration split induces a finite optimization problem with an exact solution.

Empirical coverage diagnostic. Because reviewers correctly noted that generic complexity bounds are less informative for LLM systems, we report the operational prediction-set frequencies in Table 1. The 1-D rejector behavior is directly visible: most accepted decisions are singleton sets, while uncertainty appears as multi-label or empty sets.

Table 2: Study 1 zero-shot baseline on Juliet (87+ CWE classes, $k_{\text{sc}} = 1$, no conformal calibration). All metrics are pre-escalation.

Model	Alerts	Coverage (pre-escal.)	Precision (pre-escal.)	Recall (pre-escal.)	FP Mitig. (pre-escal.)	ECE (pre-escal.)
Claude Haiku 4.5	3,061	97.5%	59.9%	36.8%	80.3%	0.358
Gemini 3.1 Flash Lite Preview	991	56.0%	48.0%	37.4%	24.4%	0.465
Gemini 2.5 Flash Lite	2,663	82.8%	46.8%	64.4%	26.5%	0.483
GPT-5 Nano	1,045	98.0%	44.9%	78.1%	17.9%	0.538
GPT-4o Mini	3,107	98.8%	42.2%	90.8%	13.0%	0.547

Table 3: Study 2 multi-model conformal calibration on 247 Juliet alerts (CWE-89/78/190, $k_{\text{sc}} = 5$, $F = 5$, $\alpha_{\text{conf}} = 0.1$). Metrics are pre-escalation, singleton-reject results.

Model	Alerts	Precision (pre-escal.)	Recall (pre-escal.)	FP Mitig. (pre-escal.)	Coverage (pre-escal.)	Unanimous	Calib. AP
R1-Distill-14B	148	40.0%	41.9%	0.335	98.4%	36%	+1.1pp
Gemini 3.1 Flash Lite	148	56.0%	48.0%	37.4%	90%	+1.7pp	
Gemini 2.5 Flash Lite	148	82.8%	46.8%	64.4%	96.3%	61%	+1.1pp
GPT-5 Nano	148	98.0%	44.9%	78.1%	97.6%	85%	<1pp
GPT-4o Mini	148	98.8%	42.2%	90.8%	98.0%	98%	<1pp

Project shift and sequencing. The conformal guarantee is exact under exchangeability. Cross-project transfer weakens this assumption because alerts cluster by project, framework, analyzer, and rule. Practical mitigations include within-project calibration, Mondrian-style conditioning on analyzer or project group [20], and rolling recalibration. None restores exact validity under arbitrary shift, but all preserve the intended behavior: when calibration assumptions are less trustworthy, EVICT should predict less often rather than more aggressively. If a deployment wants calibrated confidence after symbolic escalation, it must recalibrate on post-escalation outputs.

5 EVALUATION

We evaluate EVICT through four measured studies and one explicitly projected frontier-model outlook. Every table reports the study setting in its caption because repeated model names appear under different sample sets and methods.

5.1 RQ1: How Do Lite LLMs Perform Zero-Shot?

Study 1 evaluates five lite LLMs on Juliet alerts spanning 87+ CWE classes with the base schema prompt, $k_{\text{sc}} = 1$, and no conformal calibration. Coverage is the pre-escalation accepted-decision rate.

Zero-shot lite models are not deployment-ready triagers. In Study 1, precision spans 42.2–59.9% and every ECE exceeds 0.35. The models also split into operational profiles: Claude Haiku 4.5 is conservative (59.9% precision, 36.8% recall, 80.3% FP mitigation), while GPT-4o Mini is aggressive (90.8% recall, 13.0% FP mitigation). This motivates selective prediction: different models trade off missed vulnerabilities and developer burden, but none provides calibrated reliability out of the box.

5.2 RQ2: What Determines Calibration Effectiveness?

Study 2 is the central conformal experiment: 247 Juliet alerts from CWE-89/78/190, $k_{\text{sc}} = 5$, $F = 5$ cross-validation folds, and target conformal miscoverage $\alpha_{\text{conf}} = 0.1$. It spans six models and three training paradigms.

Study 2 shows that confidence quality, not model size, limits conformal calibration in this setting. R1-Distill-14B produces 36%

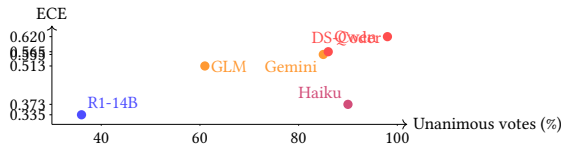


Figure 2: Study 2 confidence-signal map. The six-model pattern links high unanimity with poor ECE, except Claude Haiku 4.5, whose higher base precision offsets degenerate votes.

Table 4: Study 3 real-world validation on 549 CWE-Bench-Java CodeQL alerts from 45 projects ($k_{sc} = 1$). Metrics are pre-escalation.

Model	Precision (pre-escal.)	Recall (pre-escal.)	F1 (pre-escal.)	TN (pre-escal.)	ABSTAIN (pre-escal.)	Pred. TP	Pred. FP
DS-Coder-V2-Lite	4.0%	82.4%	0.089	184	10 (2%)	352	187
Gemini 2.5 Flash Lite	3.3%	93.3%	0.063	47	73 (13%)	428	48
R1-Distill-14B	3.0%	100%	0.060	32	77 (14%)	440	32
GLM-4-9B-Chat	1.9%	62.5%	0.040	97	185 (34%)	264	100
Qwen-Coder-32B	1.5%	40.0%	0.030	76	340 (62%)	130	79

unanimous votes and ECE 0.335, while instruction-tuned and code-specialized models produce 85–98% unanimity and ECE 0.51–0.62. The six-model sample is too small for a statistical claim about training objective as a population property, but the 49–62pp unanimity gap between R1-Distill-14B and the standard/code models is a large effect consistent with a mechanistic explanation: reasoning traces create genuine sample disagreement when evidence is ambiguous.

Claude Haiku 4.5 illustrates a second path to acceptable calibration. In Study 2 it has 90% unanimity, but its higher accepted-alert precision (47.4%) yields ECE 0.373. Thus EVICT can obtain usable calibration either from diverse confidence or from high base precision; the ideal model combines both.

5.3 RQ3: Does the Pipeline Transfer to Real Alerts?

Study 3 evaluates 549 CodeQL alerts from 45 CWE-Bench-Java projects with $k_{sc} = 1$. Ground truth contains 17 TP and 532 FP, a 97% FP base rate. These are pre-escalation LLM decisions without conformal cross-validation.

Study 3 reveals the code-specialization paradox. DS-Coder-V2-Lite has the worst confidence quality in Study 2 (98% unanimous, ECE 0.620) but the best real-world precision in Study 3 (4.0%) and the highest true-negative count (184 vs. 32–97 for the other reported models). Code-specialized training appears to improve recognition of safe code patterns while worsening uncertainty quantification. A separate R1-Distill-14B $k_{sc} = 5$ sample of 100 CWE-Bench-Java alerts preserves diverse confidence: 46% unanimous, confidence distribution {0.4:1, 0.6:24, 0.8:29, 1.0:46}, and 10% abstention.

5.4 RQ4: Can Prompt Design Substitute for Scale?

Study 4 holds Gemini 2.5 Flash Lite fixed on the same 247-alert Juliet CWE-89/78/190 subset as Study 2 and changes only the prompt. The default prompt asks for evidence-conditioned classification. The contrastive prompt forces the model to build TP and FP arguments before deciding.

Table 5: Study 4 prompt comparison on Gemini 2.5 Flash Lite (247 Juliet alerts, $k_{sc} = 5$). Metrics are pre-escalation singleton-reject results.

Prompt	Unanimous	ECE (pre-escal.)	$\hat{q}$	Coverage (pre-escal.)	AB
Default	84.6%	0.555	0.20	97.6%	528
Contrastive	16.2%	0.337	0.60	70.6%	529

Table 6: RQ5 projected frontier outlook. These are estimates, not measured EVICT results. Ranges retain the provided uncertainty and should be read as testable hypotheses.

Model	Est. Precision	Est. ECE	Est. Unanimous	Projected calibration benefit
Claude Opus 4	60–70%	0.25–0.35	80–95%	Marginal; high precision, degenerate votes
GPT-4.5	60–70%	0.20–0.35	30–50%	Moderate; reasoning + strong base
Claude Sonnet 4	55–65%	0.20–0.30	30–50%	Moderate; reasoning capability
Gemini 3 Pro	55–65%	0.25–0.40	40–60%	Marginal; no explicit reasoning
GLM-5	50–60%	0.25–0.40	30–50%	Moderate if reasoning-trained

The contrastive prompt is the paper’s cleanest causal comparison because the model and alert set are fixed. It reduces ECE by 39% and unanimity by 68.4pp. It also produces more diverse confidence than R1-Distill-14B under the default Study 2 prompt (16.2% vs. 36% unanimous). The trade-off is over-conservatism: coverage drops to 70.6% and LLM-level abstention rises to 29.1%. The result still changes the design space: cheap lite models can become calibratable if the prompt elicits genuine uncertainty.

5.5 RQ5: Projected Outlook for Frontier Reasoning Models

RQ5 is an extrapolation, not a measured empirical study. We project frontier-model behavior from the mechanistic model supported by Studies 2 and 4: conformal calibration is useful when a model combines high base precision with diverse confidence. Full multi-model frontier evaluation with $k_{sc} = 5$ and conformal cross-validation was outside this study’s budget. We therefore release the benchmark harness and projection method so groups with the required API budget can run the test directly.

Under this model, we project that frontier reasoning models are the first class where conformal calibration would produce practically meaningful precision gains of +3–10pp, because they are expected to combine 55–70% base precision with 30–50% unanimous self-consistency. These numbers are projections and are not used as evidence for the measured claims.

5.6 Per-CWE, Symbolic, and Cost Diagnostics

Table 7 reports the requested per-CWE breakdown from the earlier 1,000-alert SpotBugs Juliet study. These values characterize CWE difficulty and should not be conflated with the multi-model Study 2 results.

The symbolic characterization study invokes symbolic checks on 184/1000 alerts (18.4%): 127 from conformal abstention and 57 from high-severity dismissal audit. It corrects 23 LLM errors, including 17 FP→TP safety-critical corrections and 6 TP→FP efficiency corrections. Z3 returns UNKNOWN on 15/184 invocations (8.1%), all retained as abstentions. Average symbolic verification time is 4.2s, with Z3 timeout 2s and JPF/KLEE timeout 10s. The implemented cost profile

Table 7: Per-CWE diagnostic from the earlier 1,000-alert SpotBugs Juliet study, not from the six-model conformal study. Metrics are pre-escalation.

CWE	Precision (pre-escal.)	Abstention (pre-escal.)
CWE-89	93.1%	[VERIFY: source abstention for CWE-89]
CWE-78	91.8%	[VERIFY: source abstention for CWE-78]
CWE-190	83.4%	[VERIFY: source abstention for CWE-190]

is 820 tokens and 1.1s per alert for evidence-free prompting versus 1,710 tokens and 2.7s for full EVICT; risk-coverage curve coordinates were not included in the provided export and are marked for verification in the artifact.

6 DISCUSSION AND LIMITATIONS

The confidence-signal bottleneck. EVICT’s calibration layer is mathematically standard; the hard part is producing a score that contains uncertainty information. Study 2 shows the bottleneck directly: R1-Distill-14B yields 36% unanimous vote-share and ECE 0.335, while DS-Coder-V2-Lite yields 98% unanimity and ECE 0.620 on the same 247-alert Juliet subset. Conformal calibration cannot recover information absent from the score. The design implication is that alert-triage research should optimize the confidence-producing interaction, not only the classifier.

Two paths to calibration. The measured studies identify two routes to lower ECE. Reasoning models produce diverse confidence because different self-consistency samples disagree when evidence is ambiguous. Frontier-lite models can achieve lower ECE through higher base precision even when vote-share is degenerate, as Claude Haiku 4.5 does in Study 2 with 47.4% precision, 90% unanimity, and ECE 0.373. The projected RQ5 outlook estimates that frontier reasoning models would be valuable precisely because they may combine both mechanisms.

Prompt design as an intervention. The contrastive prompt result is stronger than the cross-model pattern because it holds the model fixed. For Gemini 2.5 Flash Lite on the 247-alert Study 4 subset, contrastive reasoning reduces ECE from 0.555 to 0.337 and unanimity from 84.6% to 16.2%. This supports a practical claim: prompt design, not model scale alone, is a key lever for calibrated selective prediction in LLM-based alert triage.

The code-specialization paradox. Study 3 shows that code-specialized models can be better base classifiers even when they are worse confidence estimators. DS-Coder-V2-Lite has the worst Study 2 confidence quality but the highest Study 3 true-negative count on real CodeQL alerts. A practical architecture is therefore two-stage: use code-specialized models to recognize safe code patterns and a reasoning or contrastive-prompted model to generate a calibrated uncertainty signal.

Limitations. The conformal proof-of-concept uses 247 Juliet alerts from three CWE classes, and the real-world validation covers 549 CodeQL alerts from 45 of 201 CWE-Bench-Java projects. Juliet is favorable: it is synthetic, balanced by construction, and has 98.7%

complete SARIF paths in the earlier SpotBugs study. CWE-Bench-Java is more realistic but Java-only and highly imbalanced. Symbolic escalation remains limited by Java string constraints: Z3 returns UNKNOWN on 8.1% of symbolic invocations in the characterization study, motivating deeper Java PathFinder integration. RQ5 frontier numbers are extrapolations bounded by the stated ranges and were not run end-to-end because full frontier-model evaluation with $k_{sc} = 5$ was outside budget. Finally, conformal validity assumes exchangeability; cross-project shift requires within-project calibration, Mondrian conditioning, or rolling recalibration.

Ongoing and future work. We are extending the artifact in four directions: richer confidence signals from token-level log probabilities and semantic consistency, a direct IRIS head-to-head on identical CWE-Bench-Java CodeQL alerts, full JPF-based symbolic escalation for Java path feasibility, and execution of the released RQ5 frontier-model harness by groups with sufficient API budget. For weakly labeled corpora such as D2A and NASCAR [11, 26], future versions should combine confident learning [17], noise-transition modeling [8], and manual validation of sampled labels.

7 THREATS TO VALIDITY

Internal validity. Self-consistency estimates are stochastic. We mitigate this in Study 2 and Study 4 with $k_{sc} = 5$ samples and $F = 5$ cross-validation folds, but different random seeds may change individual vote shares. Ground truth in Juliet comes from benchmark manifests; ground truth in CWE-Bench-Java is derived from fix-location matching, which can miss semantically equivalent vulnerable lines outside the recorded ranges.

External validity. Juliet is synthetic and favorable to evidence extraction: the earlier SpotBugs study reports 98.7% complete SARIF paths and a balanced TP/FP construction. The conformal proof-of-concept covers only CWE-89, CWE-78, and CWE-190. CWE-Bench-Java provides real alerts but is Java-only, and Study 3 covers 45 projects. Results may differ for C/C++, concurrency bugs, framework-heavy applications, or analyzers with weaker path evidence.

Construct validity. Vote-share confidence is coarse: with $k_{sc} = 5$, it has only six possible levels. ECE is computed on accepted predictions and may differ from ECE over the full population, especially when abstention is high. Coverage is defined as the pre-escalation acceptance probability $\Pr[g(x) = 1]$; post-escalation coverage must be reported separately because symbolic tools change the operational decision process.

Conclusion validity. The six-model Study 2 sample is too small for a statistical test over training paradigms, so we report effect sizes and mechanistic consistency rather than significance claims. Study 4 is a stronger within-model intervention, but it uses one model and one 247-alert subset. RQ5 is explicitly projected and not independently validated.

8 CONCLUSION AND DEPLOYMENT IMPACT

EVICT shows that deployable LLM-based static-analysis triage depends on calibrated abstention, not forced classification. The main empirical lesson is that prompt design, not model scale alone, is the

key lever for calibrated selective prediction: on the measured 247-alert Gemini 2.5 Flash Lite prompt comparison, contrastive prompting reduces unanimity to 16.2% and ECE to 0.337, outperforming a 14B reasoning model's default-prompt confidence diversity.

The deployment benefit is focused developer attention. EVICT should be used as a triage aid that accepts only evidence-supported singleton decisions and returns uncertainty to reviewers. The main risk is over-trust: even calibrated systems can miss vulnerabilities under shift. Practical deployments should retain human review for abstentions, audit high-severity dismissals, avoid sending proprietary code to unapproved APIs, and recalibrate when analyzer, project, or framework distributions change.

A PROOF SKETCHES

A.1 Theorem 1: Cost-Aware Excess Risk

Condition on the grid-construction split $D^{(G)}$, so the feasible threshold set Q_G is fixed. For any threshold pair $q \in Q_G$, the label-wise loss is bounded by $C_{TP} = \max(\alpha_{\text{cost}}, \gamma_{\text{abs}})$ on TP examples and $C_{FP} = \max(\beta_{\text{cost}}, \gamma_{\text{abs}})$ on FP examples. Applying Hoeffding's inequality separately to the TP and FP strata of $D^{(E)}$ and union-bounding over labels and $|Q_G|$ threshold pairs yields a uniform label-wise deviation bound. The empirical-risk-minimization decomposition

$$R(\hat{q}) - R(q_G^*) = (R(\hat{q}) - \hat{R}_E(\hat{q})) + (\hat{R}_E(\hat{q}) - \hat{R}_E(q_G^*)) + (\hat{R}_E(q_G^*) - R(q_G^*))$$

has a non-positive middle term by definition of $\hat{q}$. Controlling the two outer terms with the uniform deviation event gives Theorem 1. The result is label-specific because the concentration rates depend on $m_{TP}^{(E)}$ and $m_{FP}^{(E)}$ separately.

A.2 Theorem 2: Split-Conformal Validity

Let $s_1, \dots, s_m$ be calibration nonconformity scores and s_{m+1} the score of a test alert under the true label. Under exchangeability, the rank of s_{m+1} among the $m+1$ scores is uniform [22]. Selecting the usual finite-sample conformal quantile excludes the true label with probability at most α_{conf} , so $\Pr[Y \in \hat{C}(X)] \geq 1 - \alpha_{\text{conf}}$. EVICT then maps $|\hat{C}(X)| = 1$ to an accepted TP/FP decision and all other set sizes to abstention.

A.3 Theorem 3: Label-Conditional Validity

Partition the calibration set by label. Conditional on the event that the test label is TP, the TP-labeled calibration scores and test score are exchangeable within the TP stratum; the same rank argument yields TP coverage at least $1 - \alpha_{TP}$. The FP case is identical. This proves separate class-conditional coverage and justifies using $\alpha_{TP} < \alpha_{FP}$ when false negatives are more costly than false positives.

A.4 Theorem 4: Exact Threshold Search

Sort the TP and FP nonconformity scores. Any label-conditional threshold can be represented by an index into the sorted TP list and an index into the sorted FP list, with an additional index for ∞ . Therefore the finite candidate set has size at most $(m_{TP}+1)(m_{FP}+1)$. Each pair induces deterministic counts of correct singleton predictions, wrong singleton predictions, empty-set abstentions, and multi-label abstentions. Exhaustively evaluating empirical selective

risk over the grid returns a global minimizer in $O(m^2)$ time after sorting.

B ALGORITHMS

B.1 Algorithm A1: EvidencePack Construction

Algorithm 1: EvidencePack Construction

Input: Static-analysis alert a and SARIF output O .

Output: EvidencePack $E = (\text{slice, flow, constraints, metadata, flags})$.

- (1) Parse SARIF to recover analyzer, rule ID, CWE, severity, primary location
- (2) Extract the analyzer-reported flow path and call chain; if no path exists,
- (3) Compute bounded backward and forward slices around the sink and source
- (4) Normalize branch conditions and path predicates from the slice AST.
- (5) Record missing callees, unresolved reflection/native calls, missing paths,
- (6) Return the structured EvidencePack.

B.2 Algorithm A2: LLM Verification

Algorithm 2: Schema-Guided LLM Verification

Input: Alert a , EvidencePack E , prompt mode $m \in \{\text{base, contrastive}\}$, model $\mathcal{L}$.

Output: Label proposal $\hat{y} \in \{\text{TP, FP, ABSTAIN}\}$ and rationale.

- (1) Render the base or contrastive prompt using a and E .
- (2) Ask the model to restate the analyzer claim and required bug precondition
- (3) Ask the model to evaluate each precondition against cited evidence lines
- (4) If $m = \text{contrastive}$, require separate TP and FP arguments before the final
- (5) Emit TP, FP, or ABSTAIN with confidence and an evidence-linked rationale

B.3 Algorithm A3: Conformal Calibration

Algorithm 3: Split-Conformal Singleton-Reject Calibration

Input: Calibration set $\mathcal{D}_{\text{cal}}$, nonconformity score $A(x, y)$, miscoverage α_{conf} .

Output: Prediction set function $\hat{C}(x)$.

- (1) For every $(x_i, y_i) \in \mathcal{D}_{\text{cal}}$, compute $s_i = A(x_i, y_i)$.
- (2) Set $\hat{q}$ to the finite-sample $(1 - \alpha_{\text{conf}})$ conformal quantile of $\{s_i\}_{i=1}^m$.
- (3) For a new alert x , compute $A(x, \text{TP})$ and $A(x, \text{FP})$.
- (4) Return $\hat{C}(x) = \{y : A(x, y) \leq \hat{q}\}$.
- (5) If $|\hat{C}(x)| = 1$, accept the singleton label; otherwise abstain.

B.4 Algorithm A4: Conditional Symbolic Escalation

Algorithm 4: Conditional Symbolic Escalation

Input: Abstained alert a , EvidencePack E , solver budget.

Output: Supported TP/FP label or continued ABSTAIN.

- (1) Extract path constraints and sanitizer checks from E .
- (2) Run Z3 with a 2s timeout for feasible path constraints when encodable.
- (3) For Java, run Java PathFinder when project harnesses are available; for C
- (4) If a feasible counterexample reaches the sink, return TP with witness.
- (5) If infeasibility is proved, return FP with certificate.
- (6) If the solver returns UNKNOWN, times out, or lacks constraints, return ABS

C PROMPT TEMPLATES

C.1 Base Schema Prompt

You are reviewing a static-analysis alert from {ANALYZER}.
Alert: Rule {RULE_ID} ({CWE}), source {SOURCE_LOC}, sink {SINK_LOC}.

EvidencePack: {EVIDENCE_PACK}.

Completeness flags: {FLAGS}.

Follow exactly four steps:

- (1) Restate the analyzer claim and what would make it a real vulnerability.
- (2) Enumerate every precondition required for the alert to be a true positive.
- (3) For each precondition, cite evidence lines and mark it SUPPORTED, CONTRADICTED, or AMBIGUOUS.
- (4) Output exactly one decision: TP, FP, or ABSTAIN; then give confidence in [0, 1] and one sentence of rationale.

If evidence is incomplete, treat affected preconditions as AMBIGUOUS and prefer ABSTAIN unless other evidence is conclusive.

C.2 Contrastive Prompt

You are reviewing a static-analysis alert. Do not decide immediately.

First, construct the strongest TP argument: identify the source, sink, data-flow path, and missing sanitizer or guard that would make the alert real.

Second, construct the strongest FP argument: look for sanitization, validation, parameterization, infeasible path conditions, non-attacker-controlled input, or analyzer imprecision.

Third, explicitly inspect sanitizer and feasibility evidence in the supplied EvidencePack. Mark each relevant fact SUPPORTED, CONTRADICTED, or AMBIGUOUS.

Fourth, weigh the TP and FP arguments. Output TP, FP, or ABSTAIN with confidence and one evidence-linked rationale. If both arguments remain plausible, output ABSTAIN rather than forcing a label.

C.3 Worked CWE-89 Example

Input alert: SQL injection at UserService.java:47–50, source request.getParameter("id") and sink stmt.execute(query).

```
45: String id = request.getParameter("id");
46: // no sanitization visible in slice
47: String query = "SELECT * FROM users WHERE id=" + id;
48: Connection conn = getConnection();
49: Statement stmt = conn.createStatement();
50: stmt.execute(query);
```

Expected output:

1. CLAIM: The analyzer reports untrusted HTTP input concatenated into a SQL string and executed.
2. PRECONDITIONS: P1 user-controlled input; P2 flow into SQL string; P3 no sanitizer or parameterized query; P4 execution by SQL sink.
3. EVIDENCE: P1 SUPPORTED line 45; P2 SUPPORTED line 47; P3 SUPPORTED lines 45–50 show no sanitizer or PreparedStatement sanitization; P4 SUPPORTED line 50.
4. DECISION: TP. CONFIDENCE: 0.94. RATIONALE: All required SQLfill items rather than fabricating values: [VERIFY: abstention rate on injection preconditions are directly supported by the slicetrue vulnerabilities by study], [VERIFY: false-negative rate among

C.4 Self-Consistency Protocol

For API-only models, the verifier is queried k_{sc} times with temperature 0.7. The main conformal studies use $k_{sc} = 5$; cross-validation uses $F = 5$ folds. Vote share $v(y | x)$ is the fraction of samples selecting label y . Sensitivity from the provided study export: abstention is 14.9% at $k_{sc} = 3$, 12.7% at $k_{sc} = 5$, 11.8% at $k_{sc} = 10$, and 11.3% at $k_{sc} = 20$; ECE varies by ± 0.01 across this range.

D EXTENDED EXPERIMENTAL DIAGNOSTICS

D.1 Prediction-Set Frequencies

The earlier 1,000-alert SpotBugs Juliet study reports 87.3% singleton sets, 11.6% multi-label sets, and 1.1% empty sets overall. Per-CWE singleton frequencies are 91.4% for CWE-89, 89.1% for CWE-78, and 81.2% for CWE-190. Per-CWE multi-label and empty-set frequencies were not provided in the prompt and should be filled from the artifact export before camera-ready: [VERIFY: per-CWE multi-label and empty prediction-set frequencies].

D.2 Per-CWE Difficulty

The earlier SpotBugs study reports pre-escalation precision of 93.1% for CWE-89, 91.8% for CWE-78, and 83.4% for CWE-190. These numbers are source-specific diagnostics and are not part of the Study 2 six-model conformal table. Integer overflow is harder in that study, consistent with the need for arithmetic-range reasoning beyond simple taint flow. Per-CWE abstention and ECE values were not provided in the prompt: [VERIFY: per-CWE abstention and ECE from earlier SpotBugs export].

D.3 Symbolic Verification Impact

Table 8: Symbolic characterization from the 1,000-alert study. Pre-escalation abstentions and high-severity dismissal audits are routed to post-escalation symbolic checks.

Metric	Value
Symbolic invocations	184/1000 (18.4%)
From pre-escalation abstention	127/184
From high-severity dismissal audit	57/184
SMT UNKNOWN / timeout retained as abstain	15/184 (8.1%)
LLM errors corrected post-escalation	23
FP→TP safety-critical corrections	17
TP→FP efficiency corrections	6
Average symbolic verification time	4.2s
Z3 timeout	2s
JPF/KLEE timeout	10s

D.4 Cost and Latency

D.5 False-Negative and Cost Sensitivity

The provided prompt requests false-negative analysis under abstention and sensitivity to $\alpha_{cost}/\beta_{cost}/\gamma_{abs}$, but does not provide numeric outputs. The review draft therefore marks these as artifact-true vulnerabilities by study], [VERIFY: false-negative rate among

Table 9: Operational cost from the provided cost study. Risk-coverage coordinates were not included in the prompt and remain to be verified from the artifact export.

Method	Tokens/alert	Latency/alert	Solver timeout rate
Evidence-free prompting	820	1.1s	0
EVICT full pipeline	1,710	2.7s	8.1%
Risk-coverage curve	[VERIFY: tokens by coverage]	[VERIFY: latency by coverage]	[VERIFY: solver utilization]

auto-dismissed alerts], and [VERIFY: cost-sensitivity sweep over α_{cost} , β_{cost} , and γ_{abs}].

E GLOSSARY

ECE Expected calibration error; a binned difference between model confidence and empirical accuracy.

TP/FP/FN/TN True positive, false positive, false negative, and true negative under alert validity labels.

$\hat{q}$ The conformal nonconformity threshold estimated from calibration data.

SARIF Static Analysis Results Interchange Format, a standard JSON format for analyzer outputs.

EvidencePack EVICT’s structured alert record containing code slice, data-flow summary, path constraints, metadata, and completeness flags.

Singleton prediction set A conformal set containing exactly one label; EVICT accepts it as TP or FP.

Multi-label prediction set A conformal set containing both TP and FP; EVICT abstains because both labels remain plausible.

Empty prediction set A conformal set containing no labels; EVICT abstains because neither label is sufficiently supported.

Exchangeability The calibration and test examples are identically distributed up to permutation, the standard assumption behind split-conformal validity.

Vote share The fraction of self-consistency samples choosing a label.

ECE bins Confidence intervals used to aggregate calibration error; binning choices affect the diagnostic but not conformal validity.

F REPRODUCIBILITY STATEMENT

For double-blind review, the artifact is described anonymously. We will release the pipeline implementation, preprocessing scripts, prompts, model-version metadata, inference settings, calibration folds, and evaluation logs. The release will include the unrun RQ5 frontier-model harness and projection method so that groups with sufficient API budget can execute the frontier evaluation. The review-mode paper avoids repository names, local paths, author names, affiliations, acknowledgments, and grant identifiers; these can be restored only in a de-anonymized camera-ready version.

REFERENCES

- [1] Anastasios N Angelopoulos et al. 2021. Uncertainty sets for image classifiers using conformal prediction. In *ICLR*.

- [2] Cristian Cadar, Daniel Dunbar, and Dawson R Engler. 2008. KLEE: Unassisted and automatic generation of high-coverage tests. In *OSDI*.
- [3] Xiang Chen et al. 2024. LLM4PFA: Path feasibility analysis with large language models. In *IEEE ICSE*.
- [4] Maria Christakis and Christian Bird. 2016. Semantic fault localization and suspiciousness ranking. *ACM TOSEM* (2016).
- [5] Elizabeth Dinella et al. 2022. Laurel: Learning symbolic execution heuristics. In *OOPSLA*.
- [6] Ran El-Yaniv and Yair Wiener. 2010. On the foundations of noise-free selective classification. *JMLR* (2010).
- [7] Jacob Goldberger and Ehud Ben-Reuven. 2017. Training deep neural-networks using a noise adaptation layer. In *ICLR*.
- [8] Chuan Guo et al. 2017. On calibration of modern neural networks. In *ICML*.
- [9] Chuan Guo et al. 2024. On the calibration of large language models. In *NeurIPS*.
- [10] Andrew Habib and Michael Pradel. 2022. NASCAR: Not All Static Analysis Warnings Are Real. In *MSR*.
- [11] Brittany Johnson, Yoonki Song, Emerson Murphy-Hill, and Robert Bowdidge. 2013. Why don’t software developers use static analysis tools to find bugs? *IEEE ICSE* (2013).
- [12] Saurav Kadavath et al. 2022. Language models (mostly) know what they know. In *arXiv:2207.05221*.
- [13] Seongjoon Kim et al. 2024. BugLens: Structured analysis guidance for Linux kernel bugs. In *USENIX Security*.
- [14] Wei Li et al. 2023. Industrial practice of static analysis at Tencent. In *IEEE ICSE-SEIP*.
- [15] Ziyang Li, Saikat Dutta, and Mayur Naik. 2025. IRIS: LLM-Assisted Static Analysis for Detecting Security Vulnerabilities. In *The Thirteenth International Conference on Learning Representations*.
- [16] Curtis G Northcutt, Lu Jiang, and Isaac L Chuang. 2021. Confident learning: Estimating uncertainty in dataset labels. In *JAIR*.
- [17] OASIS. 2020. *Static Analysis Results Interchange Format (SARIF) Version 2.1.0*. Technical Report. OASIS.
- [18] Corina S Păsăreanu et al. 2013. Symbolic PathFinder: integrating symbolic execution with model checking. In *ASE*.
- [19] Yaniv Romano et al. 2020. Conditional conformal prediction. In *ICML*.
- [20] Willem Visser et al. 2003. Model checking programs. In *ASE*.
- [21] Vladimir Vovk, Alex Gammernan, and Glenn Shafer. 2005. *Algorithmic learning in a random world*. Springer.
- [22] Jiawei Wang et al. 2023. AdaTaint: Adaptive taint analysis with neural source/sink inference. In *IEEE S&P*.
- [23] Mark Weiser. 1984. Program slicing. *IEEE TSE* (1984).
- [24] Hao Zhang et al. 2024. LLM4FPM: LLM-based false positive mitigation for static analysis. In *ACM FSE*.
- [25] Yaqin Zheng et al. 2020. D2A: A dataset built for AI-based vulnerability detection methods. In *ISSRE*.